Module 6 Week B — Reading: Embeddings & Semantic Representations

In Module 6 Week A you built Natural Language Processing (NLP) pipelines that extract structured information from text: tokenization, normalization, named entity recognition, and entity evaluation. You can now take a raw document and pull out the names, organizations, and locations it mentions. But entities are only part of the picture. What if you want to know which two documents are most *similar* to each other? What if you want to search a corpus by meaning rather than by keyword?

To do any of that, you need to answer a fundamental question: **how do you represent text as numbers?** Machine learning models operate on numeric inputs. Every NLP task – classification, clustering, search, recommendation – requires turning words and sentences into vectors that preserve (as much as possible) the meaning of the original text.

This reading traces the full spectrum of text representations: from the simplest approach (counting words) through dense static embeddings (GloVe) to contextual embeddings (BERT/DistilBERT) that change based on surrounding words. Each approach trades off simplicity, speed, and semantic richness. By the end, you will understand when to use each representation and how to extract embeddings from a transformer model.

What success looks like: By the end of Week B, you can build TF-IDF representations and use them for similarity-based retrieval, articulate where count-based representations fall short for semantic tasks, load and query pre-trained GloVe word embeddings, extract contextual embeddings from DistilBERT using the tokenizer-model-pooling pipeline, compare static and contextual embeddings for text similarity tasks, and choose the right representation for a given NLP task.

In This Reading

1. Why Text Needs Numbers
2. Bag-of-Words
3. TF-IDF
4. Word Embeddings: The Key Insight
5. word2vec
6. GloVe
7. From Static to Contextual
8. BERT and Contextual Embeddings
9. Extracting Embeddings from Transformers
10. Choosing the Right Representation

- Troubleshooting
- Key Terms
- Summary
- What's Next

---

1. Why Text Needs Numbers

Every ML model you have built so far – linear regression, decision trees, random forests – operates on numeric feature vectors. When the input is a table of numbers (customer tenure, monthly charges, transaction counts), the pipeline is straightforward: load the data, split, train, evaluate. But text is not a table of numbers. Text is a sequence of symbols with grammar, ambiguity, and meaning that depends on context.

The **representation problem** is: how do you convert a sentence into a fixed-length numeric vector without losing the information that matters for your task?

There is no single answer. Instead, there is a spectrum of approaches, each with different properties:

Approach	Vector Type	Captures Semantics?	Context-Aware?	Speed
Bag-of-Words	Sparse, high-dimensional	No	No	Very fast
TF-IDF	Sparse, high-dimensional	Partially (word importance)	No	Very fast
Static embeddings (GloVe)	Dense, low-dimensional	Yes (distributional)	No	Fast
Contextual embeddings (BERT)	Dense, medium-dimensional	Yes (deep)	Yes	Slow

The rest of this reading walks through each approach, building from the simplest to the most powerful. By the end, you will understand not just *how* each works but *when* each is the right choice.

---

2. Bag-of-Words

The Simplest Representation

The **bag-of-words (BoW)** model represents a document as a vector of word counts. The process:

1. Build a **vocabulary** from all unique words across all documents in the corpus.
2. For each document, create a vector with one entry per vocabulary word.
3. Each entry records how many times that word appears in the document (term frequency) or simply whether it appears (binary).

Consider three sentences:

- "climate change affects oceans"
- "ocean temperatures are rising"
- "climate policy changes slowly"

The vocabulary is: {affects, are, change, changes, climate, ocean, oceans, policy, rising, slowly, temperatures}. Each sentence becomes a vector of length 11. Most entries are zero.

BoW with scikit-learn

```
from sklearn.feature_extraction.text import CountVectorizer

texts = [
    "climate change affects oceans",
    "ocean temperatures are rising",
    "climate policy changes slowly"
]

vectorizer = CountVectorizer()
bow_matrix = vectorizer.fit_transform(texts)

print(f"Shape: {bow_matrix.shape}")           # (3, 11)
print(f"Vocabulary: {vectorizer.get_feature_names_out()}")
print(f"Dense matrix:\n{bow_matrix.toarray()}")
```

`CountVectorizer` handles tokenization, lowercasing, and vocabulary construction automatically. The result is a **sparse matrix** – most entries are zero, so scikit-learn stores only the nonzero values to save memory.

Limitations of Bag-of-Words

BoW has fundamental problems that no amount of tuning can fix:

1. **No word order.** "Dog bites man" and "man bites dog" produce identical BoW vectors. The representation discards all sequential structure.
2. **No semantics.** The words "car" and "automobile" are treated as completely unrelated – they occupy different dimensions of the vector with no connection between them.
3. **Huge, sparse vectors.** A corpus with 50,000 unique words produces vectors of length 50,000, almost entirely zeros. This is computationally wasteful and makes similarity comparisons unreliable in high-dimensional space (the "curse of dimensionality").
4. **Common words dominate.** Words like "the", "is", and "and" appear in almost every document, inflating their counts without carrying useful information.

BoW is a starting point, not a destination. Its simplicity makes it useful as a baseline, and its speed makes it practical for very large corpora where more sophisticated methods are too slow.

---

3. TF-IDF

Improving on Raw Counts

TF-IDF (Term Frequency – Inverse Document Frequency) addresses the "common words dominate" problem. Instead of raw counts, each word gets a score that reflects how important it is to a specific document relative to the corpus:

- **Term Frequency (TF):** How often the word appears in this document. More frequent = higher TF.
- **Inverse Document Frequency (IDF):** How rare the word is across all documents. Rarer = higher IDF. Computed as log(total documents / documents containing the word).
- **TF-IDF = TF x IDF:** Words that are frequent in one document but rare across the corpus get high scores. Words that appear everywhere (like "the") get low scores.

TF-IDF with scikit-learn

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer()
tfidf_matrix = vectorizer.fit_transform(texts)

print(f"Shape: {tfidf_matrix.shape}")
print(f"Feature names: {vectorizer.get_feature_names_out()}")
```

`TfidfVectorizer` combines tokenization, vocabulary construction, and TF-IDF weighting in one step. The output is still a sparse matrix, but now the values are continuous scores rather than integer counts.

Computing Similarity with TF-IDF

Once documents are represented as TF-IDF vectors, you can compute **cosine similarity** to measure how similar two documents are:

```
from sklearn.metrics.pairwise import cosine_similarity

similarity_matrix = cosine_similarity(tfidf_matrix)
print(similarity_matrix)
```

Cosine similarity measures the angle between two vectors, ignoring their magnitude. Values range from 0 (completely different) to 1 (identical direction). Two documents about climate that share vocabulary will have high cosine similarity even if one is longer than the other.

What TF-IDF Still Cannot Do

TF-IDF is a significant improvement over BoW: it surfaces informative words and downweights noise. But it shares BoW's fundamental limitations:

- **Still no word order.** "Dog bites man" and "man bites dog" have the same TF-IDF representation.
- **Still no semantics.** "Car" and "automobile" remain unrelated. Two documents about the same topic using different vocabulary will have low similarity.
- **Still sparse and high-dimensional.** The vectors are the same size as the vocabulary.

TF-IDF is the right choice when keyword overlap is a good proxy for relevance – information retrieval, document search, and document classification tasks where the vocabulary itself is informative. When you need to capture meaning beyond shared words, you need embeddings.

---

4. Word Embeddings: The Key Insight

The Distributional Hypothesis

The breakthrough insight behind word embeddings is simple: **words that appear in similar contexts have similar meanings.** This is the **distributional hypothesis**, articulated by linguist J.R. Firth in 1957: "You shall know a word by the company it keeps."

Consider these sentences:

- "The **king** ruled the kingdom wisely."
- "The **queen** ruled the kingdom wisely."
- "The **monarch** ruled the kingdom wisely."

The words "king," "queen," and "monarch" appear in nearly identical contexts. A model trained on large amounts of text can learn from these patterns that these words are semantically related – without anyone telling it what the words mean.

From Sparse to Dense

Instead of one dimension per vocabulary word (sparse, 50,000+ dimensions), word embeddings represent each word as a **dense vector** with far fewer dimensions – typically 50 to 300. These dimensions are not interpretable in isolation (dimension 17 does not mean "royalty"), but the geometric relationships between vectors capture semantic relationships:

- Words with similar meanings are close together in the embedding space.
- Relationships between words are encoded as vector differences: the direction from "king" to "queen" is approximately the same as the direction from "man" to "woman."

This is a fundamentally different kind of representation. BoW and TF-IDF treat each word as an independent symbol. Embeddings treat words as points in a continuous space where distance corresponds to meaning.

---

5. word2vec

How word2vec Learns Embeddings

word2vec (Mikolov et al., 2013) is the model that popularized word embeddings. It learns dense word vectors by training a shallow neural network on a simple prediction task. There are two variants:

- **Skip-gram:** Given a center word, predict the surrounding context words. Input: "king." Output: "the," "ruled," "kingdom," "wisely." The network learns word vectors that make correct context predictions.
- **CBOW (Continuous Bag of Words):** Given the context words, predict the center word. Input: "the," "ruled," "kingdom," "wisely." Output: "king."

In both cases, the word vectors are the learned parameters of the network. After training on billions of words, similar words end up with similar vectors because they predict (and are predicted by) similar contexts.

The Famous Analogies

word2vec's most celebrated property is **vector arithmetic for analogies:**

king - man + woman ≈ queen

This works because the model learns that the relationship between "king" and "man" (royalty minus male) is similar to the relationship between "queen" and "woman" (royalty minus female). The vector difference encodes the concept of gender, and adding it to a different starting point produces the expected result.

Other classic examples:

- Paris - France + Italy ≈ Rome (capital-country relationships)
- walked - walking + swimming ≈ swam (tense relationships)

Limitations of word2vec

Despite its elegance, word2vec has important limitations:

1. **One vector per word.** The word "bank" gets a single embedding whether it means "river bank" or "financial bank." The model has no way to represent polysemy (multiple meanings for the same word).
2. **Analogy cherry-picking.** The "king - queen" example is carefully selected. Many analogies fail or produce unexpected results. Do not assume word2vec analogies are reliable for arbitrary word pairs.
3. **Training requirements.** word2vec must be trained on a large corpus to produce useful embeddings. Pre-trained vectors (trained on billions of words from Google News or Wikipedia) are typically used rather than training from scratch.

You will not train word2vec models in this course. The conceptual understanding matters: word2vec demonstrated that useful semantic representations can be learned from text alone, without manual annotation.

A note on fastText

fastText (Bojanowski et al., 2017) is a word2vec extension that represents each word as a sum of character n-gram vectors. The practical payoff is **out-of-vocabulary (OOV) handling**: where word2vec returns nothing for a word it has never seen, fastText can compose a reasonable vector from the word's subword pieces. This matters for morphologically rich languages and domain text full of rare or compound words.

You will not work with fastText in this course. GloVe is the hands-on representative for static embeddings — once you understand the load-and-query pattern with one of these models, the others are interchangeable.

---

6. GloVe

Global Vectors for Word Representation

GloVe (Pennington et al., 2014) is another method for learning word embeddings. While word2vec uses local context windows (nearby words), GloVe uses **global word co-occurrence statistics** – how often each pair of words appears together across the entire corpus. The resulting vectors capture the same kinds of semantic relationships as word2vec, with comparable quality.

In practice, GloVe and word2vec produce similar results. The distinction matters for researchers; for practitioners, the choice usually comes down to which pre-trained vectors are available and convenient.

Pre-trained GloVe Vectors

The GloVe project provides pre-trained vectors in several sizes:

Name	Dimensions	Vocabulary	Trained on
glove.6B.50d	50	400K words	Wikipedia + Gigaword (6B tokens)
glove.6B.100d	100	400K words	Wikipedia + Gigaword
glove.6B.200d	200	400K words	Wikipedia + Gigaword
glove.6B.300d	300	400K words	Wikipedia + Gigaword

Higher dimensions capture more nuance but require more memory and compute. For this course, you will use a pre-extracted subset: **50,000 words at 50 dimensions** (`glove_50k_50d.txt`, ~20 MB) — the 50,000 most frequent words from `glove.6B.50d`. Large enough to keep out-of-vocabulary rates low on general English text while remaining quick to load.

Loading GloVe Vectors

GloVe vectors are distributed as plain text files. Each line contains a word followed by its vector components, separated by spaces:

```
the 0.418 0.24968 -0.41242 0.1217 ...
of 0.70853 0.57088 -0.4716 0.18048 ...
```

Loading them into Python:

```
import numpy as np

def load_glove(filepath):
    """Load GloVe vectors from a text file into a dictionary."""
    embeddings = {}
    with open(filepath, "r", encoding="utf-8") as f:
        for line in f:
            parts = line.strip().split()
            word = parts[0]
            vector = np.array(parts[1:], dtype=np.float32)
            embeddings[word] = vector
    return embeddings

glove = load_glove("data/glove_50k_50d.txt")
print(f"Loaded {len(glove)} words")
print(f"Vector shape: {glove['climate'].shape}")  # (50,)
```

The result is a dictionary mapping words to numpy arrays. Each array has 50 elements (for 50-dimensional vectors).

Computing Cosine Similarity

Cosine similarity measures how similar two vectors are by computing the cosine of the angle between them:

cosine_similarity(A, B) = (A dot B) / (		A		*		B		)

Values range from -1 (opposite directions) to 1 (same direction). For word embeddings, higher cosine similarity means more semantically similar words.

```
def cosine_similarity(vec_a, vec_b):
    """Compute cosine similarity between two vectors."""
    dot_product = np.dot(vec_a, vec_b)
    norm_a = np.linalg.norm(vec_a)
    norm_b = np.linalg.norm(vec_b)
    if norm_a == 0 or norm_b == 0:
        return 0.0
    return dot_product / (norm_a * norm_b)

sim = cosine_similarity(glove["king"], glove["queen"])
print(f"king-queen similarity: {sim:.3f}")  # Expected: ~0.65-0.75
```

Finding Nearest Neighbors

Given a query word, you can find the most similar words by computing cosine similarity against every word in the vocabulary:

```
def nearest_neighbors(word, embeddings, n=5):
    """Find the top-n most similar words to the query."""
    query_vec = embeddings[word]
    similarities = {}
    for w, vec in embeddings.items():
        if w == word:
            continue
        similarities[w] = cosine_similarity(query_vec, vec)
    sorted_words = sorted(similarities.items(), key=lambda x: x[1], reverse=True)
    return sorted_words[:n]

neighbors = nearest_neighbors("climate", glove)
for word, sim in neighbors:
    print(f"  {word}: {sim:.3f}")
```

Representing Documents with GloVe

GloVe provides vectors for individual words. To represent an entire document, a common approach is to **average the word vectors** for all words in the document:

```
def text_to_glove(text, embeddings):
    """Compute the average GloVe embedding for a text (50-d)."""
    words = text.lower().split()
    vectors = [embeddings[w] for w in words if w in embeddings]
    if not vectors:
        return np.zeros(50)
    return np.mean(vectors, axis=0)
```

This approach has a key limitation: it ignores word order (just like BoW) and treats all words equally. It also must handle **out-of-vocabulary (OOV) words** – words not in the pre-trained vocabulary. The function above simply skips OOV words. If all words are OOV, it returns a zero vector.

Edge case: Domain-specific text (climate science, medical literature, legal documents) often contains terms not in general-purpose embeddings. The word "IPCC" may not appear in a GloVe vocabulary trained on general text. When working with specialized domains, check your OOV rate – if it is high (more than 20-30%), consider using domain-specific embeddings or a transformer model that handles subword tokenization.

---

7. From Static to Contextual

The Polysemy Problem

Static embeddings (word2vec, GloVe) assign **one vector per word**, regardless of context. This creates a fundamental problem for words with multiple meanings:

- "The **bank** of the river was muddy." (geographic feature)
- "She deposited money at the **bank**." (financial institution)
- "The pilot had to **bank** the aircraft sharply." (tilting action)

All three uses of "bank" get the exact same GloVe vector – a single point in embedding space that tries to represent all meanings simultaneously. The result is a compromise that does not accurately represent any single meaning.

This is not just a problem for obvious homonyms. Many common words shift meaning subtly depending on context:

- "The **model** performed well on the test set." (ML model)
- "The **model** walked down the runway." (fashion model)
- "She built a **model** of the bridge." (physical replica)

The Transformer Solution

Contextual embeddings solve the polysemy problem by making the representation of each word depend on the entire input sequence. Instead of looking up a fixed vector, a contextual model processes the full sentence and produces a different vector for each word based on its specific context.

The key technology is the **transformer architecture** (Vaswani et al., 2017), which uses **self-attention** to allow each word to "attend to" every other word in the input. When computing the representation of "bank" in "river bank was muddy," the model attends heavily to "river" and "muddy," producing a vector that represents the geographic meaning. In "deposited money at the bank," it attends to "deposited" and "money," producing a different vector for the financial meaning.

This is the revolution that transformed NLP starting in 2018.

---

8. BERT and Contextual Embeddings

What BERT Is

BERT (Bidirectional Encoder Representations from Transformers) is a transformer model pre-trained on large amounts of text using two tasks:

1. **Masked Language Modeling (MLM):** Randomly mask 15% of tokens in a sentence, then predict the masked tokens from context. This forces the model to understand the relationships between all words in a sentence.
2. **Next Sentence Prediction (NSP):** Given two sentences, predict whether the second follows the first in the original text. This teaches the model about inter-sentence relationships.

After pre-training on billions of words (English Wikipedia + BookCorpus), BERT has learned deep contextual representations of language. Each token's output embedding is a function of *all* other tokens in the input, captured through multiple layers of self-attention.

DistilBERT: The Practical Alternative

Full BERT (`bert-base-uncased`) has 110 million parameters. **DistilBERT** (`distilbert-base-uncased`) is a smaller version with 66 million parameters – 40% smaller and about 60% faster, while retaining approximately 97% of BERT's language understanding.

For this course, you will use DistilBERT. The quality tradeoff is minimal for most tasks, and the speed improvement matters when running on CPU (which all of you are).

First-run note: The first time you load DistilBERT, the Hugging Face `transformers` library downloads the model weights (~250MB). This download happens once and is cached locally in `~/.cache/huggingface/`. Subsequent loads are fast. If you are on a slow connection, plan for this initial download.

The Embedding Pipeline

Extracting contextual embeddings from a transformer model follows a consistent pipeline:

```
text --> tokenizer --> input IDs + attention mask --> model --> hidden states -->
```

Each step transforms the input:

1. **Tokenizer:** Converts text to token IDs that the model understands. BERT uses **WordPiece tokenization** – it may split words into subword pieces (e.g., "embeddings" becomes ["em", "##bed", "##ding", "##s"]).
2. **Model forward pass:** Processes the token IDs through multiple transformer layers. The output is a tensor of **hidden states** – one vector per token per layer.
3. **Pooling:** Reduces the per-token hidden states to a single vector representing the entire input. Common strategies:
    - **Mean pooling:** Average the hidden states across all tokens. Generally produces the best sentence embeddings.
    - **CLS token:** Use the hidden state of the special `[CLS]` token (always the first token). This was BERT's original design for classification tasks, but it is not always the best choice for similarity.

---

9. Extracting Embeddings from Transformers

Loading the Model

```
from transformers import AutoTokenizer, AutoModel
import torch

# Load tokenizer and model
tokenizer = AutoTokenizer.from_pretrained("distilbert-base-uncased")
model = AutoModel.from_pretrained("distilbert-base-uncased")

# Set model to evaluation mode (disables dropout)
model.eval()
```

`AutoTokenizer` and `AutoModel` automatically detect the correct class for the model name. `model.eval()` is critical – it disables dropout layers that would introduce randomness during inference.

Tokenizing Text

```
text = "Climate change affects global food production systems."

inputs = tokenizer(text, return_tensors="pt", truncation=True, max_length=512)
print(f"Input IDs shape: {inputs['input_ids'].shape}")
print(f"Tokens: {tokenizer.convert_ids_to_tokens(inputs['input_ids'][0])}")
```

`return_tensors="pt"` returns PyTorch tensors (required by the model). `truncation=True` ensures texts longer than `max_length` (512 tokens for BERT) are truncated rather than causing an error.

The tokenizer adds special tokens: `[CLS]` at the beginning and `[SEP]` at the end. These are part of the model's expected input format.

Extracting Hidden States

```
with torch.no_grad():
    outputs = model(**inputs)

hidden_states = outputs.last_hidden_state
print(f"Hidden states shape: {hidden_states.shape}")
# Shape: (1, num_tokens, 768)
# 1 = batch size, num_tokens = sequence length, 768 = embedding dimension
```

`torch.no_grad()` disables gradient computation – you are extracting embeddings, not training the model. This saves memory and computation.

`outputs.last_hidden_state` is a tensor of shape `(batch_size, sequence_length, hidden_dim)`. For DistilBERT, `hidden_dim` is 768. Each token in the input has its own 768-dimensional contextual representation.

Mean Pooling

To get a single vector for the entire text, average across the token dimension:

```
def mean_pool(hidden_states, attention_mask):
    """Mean pool hidden states, accounting for padding tokens."""
    # Expand attention mask to match hidden state dimensions
    mask = attention_mask.unsqueeze(-1).expand(hidden_states.size()).float()
    # Zero out padding token hidden states
    masked_hidden = hidden_states * mask
    # Sum and divide by number of real (non-padding) tokens
    summed = masked_hidden.sum(dim=1)
    counts = mask.sum(dim=1)
    return summed / counts

embedding = mean_pool(hidden_states, inputs["attention_mask"])
embedding = embedding.squeeze().numpy()
print(f"Sentence embedding shape: {embedding.shape}")  # (768,)
```

The attention mask ensures that padding tokens (added to make all inputs the same length in a batch) do not contribute to the average. For single-text inputs, there is no padding, but including the mask makes the code correct for batched inputs as well.

Putting It All Together

```
def extract_bert_embedding(text, tokenizer, model):
    """Extract a sentence embedding from DistilBERT using mean pooling."""
    inputs = tokenizer(text, return_tensors="pt", truncation=True, max_length=512
    with torch.no_grad():
        outputs = model(**inputs)
    hidden_states = outputs.last_hidden_state
    embedding = mean_pool(hidden_states, inputs["attention_mask"])
    return embedding.squeeze().numpy()

emb1 = extract_bert_embedding("Climate change threatens biodiversity.", tokenizer
emb2 = extract_bert_embedding("Global warming endangers species.", tokenizer, moc
emb3 = extract_bert_embedding("The stock market closed higher today.", tokenizer,

# Compute similarities
from sklearn.metrics.pairwise import cosine_similarity as sklearn_cosine

sim_12 = sklearn_cosine([emb1], [emb2])[0][0]
sim_13 = sklearn_cosine([emb1], [emb3])[0][0]
print(f"climate-warming similarity: {sim_12:.3f}")  # High
print(f"climate-stock similarity: {sim_13:.3f}")     # Low
```

Notice that "climate change threatens biodiversity" and "global warming endangers species" share no words but produce highly similar embeddings – the model understands they mean nearly the same thing. This is what TF-IDF and GloVe cannot do.

Computing Sentence Similarity

Once you have embeddings for multiple texts, computing a pairwise similarity matrix is straightforward:

```
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity

# Compute embeddings for all texts
embeddings = np.array([extract_bert_embedding(t, tokenizer, model) for t in texts

# Pairwise similarity matrix
similarity_matrix = cosine_similarity(embeddings)
```

Performance note: Extracting DistilBERT embeddings requires a forward pass through the model for each text. On CPU, this takes roughly 0.05-0.1 seconds per short text. For 200 texts, expect 10-20 seconds. For larger corpora, batching (processing multiple texts simultaneously) significantly improves throughput, but single-text extraction is sufficient for this course.

---

10. Choosing the Right Representation

The Decision Framework

There is no universally best text representation. The right choice depends on your task, data, and constraints:

Criterion	BoW / TF-IDF	GloVe (static)	BERT (contextual)
Best for	Keyword matching, document classification, search by exact terms	Word similarity, analogy tasks, simple document similarity	Semantic similarity, tasks where word sense matters, short text comparison
Handles synonyms	No	Partially (similar words are nearby)	Yes (contextual understanding)
Handles polysemy	No	No	Yes (different vectors per context)
Speed	Very fast	Fast (lookup + average)	Slow (forward pass per text)
Memory	Sparse matrices (efficient storage)	~50-300 floats per word	~768 floats per text + model in memory
Vocabulary	Exact match only	Limited to pre-trained vocabulary	Subword tokenization handles any input

Decision Rules of Thumb

1. **Start with TF-IDF** for document classification and keyword-based search. It is fast, interpretable, and surprisingly effective. If TF-IDF performs well enough for your task, you may not need embeddings.

2. **Use GloVe** when you need word-level similarity and context is not critical. GloVe is excellent for finding related words, building simple recommendation systems, and tasks where "similar meaning" is more important than "exact same words."

3. **Use contextual embeddings** when semantic meaning matters – especially for short texts where word sense is critical, or for tasks like semantic search where "find documents about the same concept" is more important than "find documents with the same words."

4. **Consider compute constraints.** On CPU, extracting BERT embeddings for thousands of documents takes minutes to hours. TF-IDF processes millions of documents in seconds. If your corpus is very large and you need real-time results, TF-IDF may be the pragmatic choice even if BERT would be more accurate.

Preview: Sentence Embedding Models (Module 8)

In Module 8, you will use a specialized library that provides pre-trained models specifically optimized for sentence similarity. These models are trained on sentence pairs (unlike BERT, which was trained on masked tokens) and produce better sentence embeddings out of the box. The manual extraction pipeline you learn this week — tokenize, forward pass, pool — is exactly what that library automates. Understanding the manual approach prepares you to use and debug the production tool.

Troubleshooting

Model download hangs or fails. The first time you load `distilbert-base-uncased`, the library downloads ~250MB of model weights. If the download fails, check your internet connection and retry. The files are cached in `~/.cache/huggingface/`. If you need to re-download, delete that directory. On shared machines, the cache location can be changed with the `HF_HOME` environment variable.

`RuntimeError: expected scalar type Float but found Half`. This happens if your PyTorch installation defaults to half-precision. Ensure you are using `torch` CPU build and that `model.eval()` is called before inference.

Tokenizer returns unexpected token IDs. BERT tokenizers use WordPiece subword tokenization. "Embeddings" becomes `["em", "##bed", "##ding", "##s"]`. This is expected behavior, not an error. The `##` prefix indicates a continuation of the previous token.

GloVe file loading is slow. The provided `glove_50k_50d.txt` is ~20 MB and loads in a few seconds. If you download larger GloVe files (e.g., the full 400K-word set at ~1GB), loading into a Python dictionary takes 30-60 seconds. For large files, consider using libraries like `gensim` that provide optimized loaders.

Cosine similarity returns NaN. This happens when one or both vectors have zero norm (all zeros). Check for OOV-only texts when averaging GloVe vectors – if every word in the text is out-of-vocabulary, the average is a zero vector.

BERT truncates my text. BERT/DistilBERT has a maximum input length of 512 tokens. If your text is longer, the tokenizer truncates it (when `truncation=True`). For long documents, consider splitting into chunks and averaging the chunk embeddings, or using only the first 512 tokens (which often contain the most important information).

Key Terms

Bag-of-words (BoW) – A text representation where each document is a vector of word counts (or binary presence indicators), one dimension per vocabulary word. Ignores word order.

Term frequency (TF) – The number of times a word appears in a document. Higher frequency suggests greater relevance to that document.

Document frequency (DF) – The number of documents in the corpus that contain a given word. Words with high DF appear in many documents.

Inverse document frequency (IDF) – A weight computed as log(total documents / document frequency). Rare words get high IDF; common words get low IDF.

TF-IDF – Term Frequency times Inverse Document Frequency. A weighting scheme that highlights words important to a specific document relative to the corpus.

Sparse vector – A vector where most elements are zero. BoW and TF-IDF produce sparse vectors because most words do not appear in most documents.

Dense vector – A vector where most or all elements are nonzero. Word embeddings are dense vectors, typically 50-768 dimensions.

Dimensionality – The number of elements in a vector. BoW vectors have dimensionality equal to the vocabulary size (often 10,000+). Embedding vectors have much lower dimensionality (50-768).

Word embedding – A dense, low-dimensional vector representation of a word, learned from text data, where geometric proximity corresponds to semantic similarity.

Embedding space – The vector space defined by word or sentence embeddings, where distance and direction encode semantic relationships.

Embedding dimension – The number of elements in an embedding vector (e.g., 50 for GloVe-50d, 768 for DistilBERT).

word2vec – A method for learning word embeddings using shallow neural networks trained on word prediction tasks (skip-gram or CBOW).

Skip-gram – A word2vec training objective where the model predicts surrounding context words given a center word.

CBOW (Continuous Bag of Words) – A word2vec training objective where the model predicts a center word given surrounding context words.

GloVe (Global Vectors for Word Representation) – A method for learning word embeddings from global word co-occurrence statistics. Produces static embeddings similar in quality to word2vec.

Co-occurrence matrix – A matrix recording how often each pair of words appears near each other in a corpus. GloVe is trained on factorizing this matrix.

Pre-trained embeddings – Word or sentence embeddings trained on large corpora (billions of words) and distributed for reuse. Avoids the need to train embeddings from scratch.

Cosine similarity – A similarity measure between two vectors, computed as their dot product divided by the product of their magnitudes. Ranges from -1 to 1.

Nearest neighbors (in embedding space) – The words or documents whose embedding vectors are closest to a query vector by cosine similarity (or other distance metric).

Static embedding – An embedding where each word has a single fixed vector regardless of context. GloVe and word2vec produce static embeddings.

Contextual embedding – An embedding where each word's vector depends on the surrounding words. BERT and DistilBERT produce contextual embeddings.

BERT (Bidirectional Encoder Representations from Transformers) – A transformer-based model pre-trained on masked language modeling and next sentence prediction, producing deep contextual word representations.

DistilBERT – A distilled (smaller, faster) version of BERT with 40% fewer parameters and ~97% of BERT's language understanding quality.

Masked language modeling (MLM) – A pre-training task where random tokens are masked and the model predicts them from context. BERT's primary training objective.

Transformer – A neural network architecture based on self-attention mechanisms, enabling each token's representation to incorporate information from all other tokens in the input.

Tokenizer – A component that converts raw text into a sequence of token IDs (integers) that a model can process. BERT tokenizers use WordPiece subword tokenization.

Input IDs – The integer token IDs produced by a tokenizer, representing each token (word or subword) as an index into the model's vocabulary.

Attention mask – A binary tensor indicating which tokens are real input (1) and which are padding (0). Used to prevent the model from attending to padding tokens.

Hidden states – The internal representations produced by each layer of a transformer model. The last hidden state is commonly used for extracting embeddings.

CLS token – A special token (`[CLS]`) added at the beginning of BERT's input. Its hidden state is sometimes used as a sentence-level representation.

Mean pooling – Averaging token-level hidden states across the sequence dimension to produce a single vector representing the entire input. Generally preferred over CLS for sentence similarity.

Sentence embedding – A fixed-length vector representing an entire sentence or short text, obtained by pooling token-level representations.

Transfer learning – Using a model pre-trained on one task (e.g., masked language modeling) as a starting point for a different task (e.g., text classification). BERT is a transfer learning model.

Pre-trained model – A model whose parameters have been learned on a large dataset and can be reused for downstream tasks, either directly (feature extraction) or after fine-tuning.

Fine-tuning – Continuing to train a pre-trained model on a new, task-specific dataset. Adjusts the model's parameters to the new task while retaining knowledge from pre-training. Covered conceptually here; practice in later modules.

Out-of-vocabulary (OOV) – A word that does not appear in the embedding vocabulary. Static embeddings (GloVe) have no representation for OOV words. Transformer tokenizers handle OOV by splitting words into known subword pieces.

Summary

Text representation is the foundation of every NLP task. This reading traced the full spectrum:

Bag-of-words and TF-IDF are count-based representations. They are fast, simple, and effective for keyword-matching tasks. Their fundamental limitation is that they treat words as independent symbols with no semantic connection – "car" and "automobile" are as different as "car" and "banana."

Static word embeddings (word2vec, GloVe) learn dense vectors from word co-occurrence patterns. Semantically similar words have similar vectors, enabling tasks like word similarity and analogy. Their limitation is polysemy – each word gets one vector regardless of context.

Contextual embeddings (BERT, DistilBERT) produce different vectors for the same word depending on its context. They capture deep semantic meaning and handle polysemy naturally. Their limitation is compute cost: a forward pass through the model is required for each text.

The right choice depends on your task. TF-IDF for keyword search. GloVe for word-level similarity. Contextual embeddings for semantic similarity where meaning matters more than vocabulary overlap. In practice, start simple and add complexity only when it improves results.

What's Next

The integration task combines everything from both weeks: Week A's NER extraction and Week B's embedding representations into a single semantic search pipeline. You will build a system that takes a query, finds the most semantically similar documents, and enriches the results with extracted entities – a preview of the retrieval-augmented systems you will build in Module 8.

© 2026 LevelUp Economy. All rights reserved. Unauthorized reproduction or distribution of this material is prohibited.